Build a RAG AI Agent in 30 Minutes

In this tutorial, we'll build a **Retrieval-Augmented Generation (RAG)** agent. But first, let's define what it is.

A normal chatbot answers questions based on memory or keyword-based logic coded into it.

A RAG chatbot, on the other hand, retrieves information from documents before generating a response, allowing it to provide more accurate and context-aware answers.

This app has two parts:

- **Backend (Python):** Reads PDFs, chunks text, creates embeddings, searches them, and generates answers.
- **Frontend (React):** A clean chat box like a customer-support widget.

Example usecase:

User: “How do I file a claim?”
Bot: searches PDFs → returns the exact answer.

Tech Stack

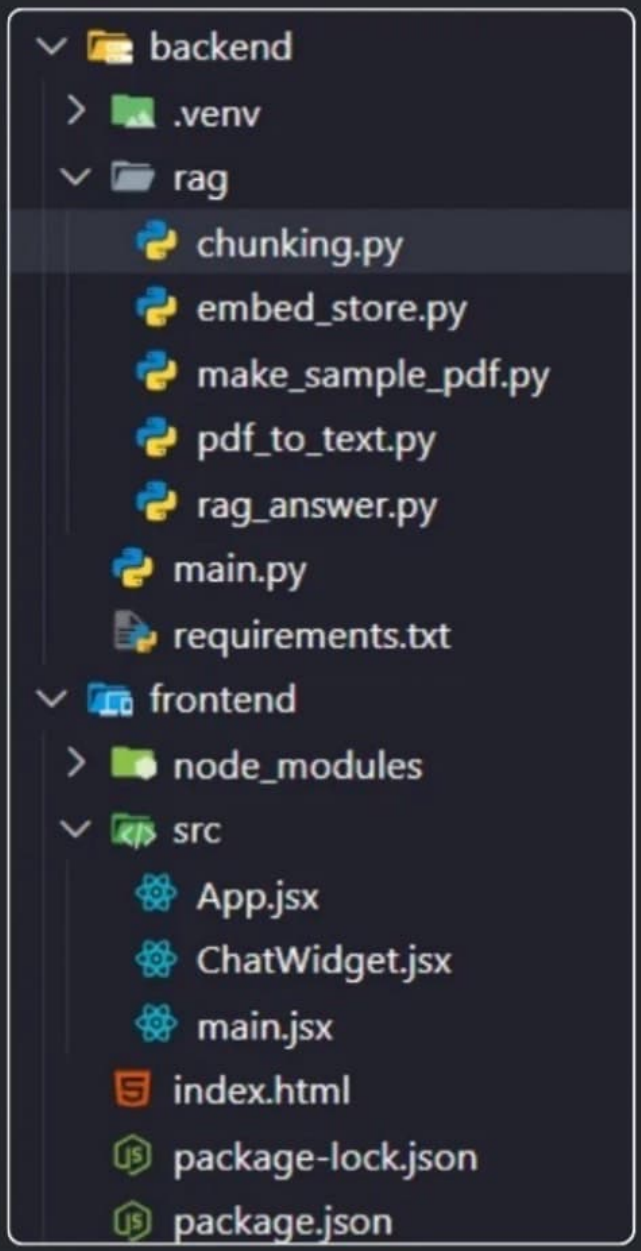
Backend:

- FastAPI – API server
- PyPDF – read PDFs
- Token chunking + overlap
- text-embedding-3-small
- FAISS – vector database
- OpenAI Responses API – chat

Frontend:

- React chat widget

Folder Structure



Step 1: Create sample PDF

Use a fake FAQ PDF to test the full pipeline.

make_sample_pdf.py

```python
from reportlab.pdfgen import canvas

def make_pdf(path="backend/data/knowledge.pdf"):
    c = canvas.Canvas(path)
    t = c.beginText(40, 750)
    t.textLine("Q: How do I file a claim?")
    t.textLine("A: Call our claims line or use the portal.")
    c.drawText(t)
    c.save()
```

Step 2: Read PDF text

In this step, we extract the text from each page of the PDF and join it into one long string. Then we do small cleanup so chunking becomes cleaner.

pdt_to_text.py

```python
from pypdf import PdfReader

def pdf_to_text(pdf_path: str) -> str:
    reader = PdfReader(pdf_path)
    pages = []
    for page in reader.pages:
        pages.append(page.extract_text() or "")
    text = "\n".join(pages)
    text = text.replace("\r", "\n")
    text = "\n".join([line.strip() for line in text.split("\n")
    if line.strip()])
    return text
```

Step 3: Chunking (most important)

Break large text into small overlapping pieces.

chunking.py

```python
import tiktoken

def chunk_text(text, size=450, overlap=80):
    enc = tiktoken.get_encoding("cl100k_base")
    tokens = enc.encode(text)

    chunks, start = [], 0
    while start < len(tokens):
        end = start + size
        chunks.append(enc.decode(tokens[start:end]))
        start = end - overlap
    return chunks
```

Step 4: Embeddings + FAISS

Turn chunks into vectors and store.

embed_store.py

```python
import json
import numpy as np
import faiss
from openai import OpenAI

client = OpenAI()
EMBED_MODEL = "text-embedding-3-small"

def embed_texts(texts):
    resp = client.embeddings.create(model=EMBED_MODEL, input=texts)
    vectors = [d.embedding for d in resp.data]
    arr = np.array(vectors, dtype="float32")
    faiss.normalize_L2(arr)
    return arr

def build_and_save_index(chunks, index_path, meta_path):
    vectors = embed_texts(chunks)
    dim = vectors.shape[1]
    index = faiss.IndexFlatIP(dim)
    index.add(vectors)

    faiss.write_index(index, index_path)
    with open(meta_path, "w", encoding="utf-8") as f:
        json.dump({"chunks": chunks}, f, ensure_ascii=False, indent=2)

def load_index(index_path, meta_path):
    index = faiss.read_index(index_path)
    with open(meta_path, "r", encoding="utf-8") as f:
        meta = json.load(f)
    return index, meta["chunks"]
```

Step 5: Retrieve + Generate

Search relevant chunks, then answer using only them.

rag_answer.py

```python
def retrieve(query, index, chunks, k=4):
    qvec = embed_texts([query])
    _, ids = index.search(qvec, k)
    return [chunks[i] for i in ids[0] if i != -1]

def generate_answer(q, ctx):
    context = "\n".join(ctx)
    res = client.responses.create(
        model="gpt-5.2",
        instructions="Use only the context.",
        input=f"{context}\n\nQuestion:{q}"
    )
    return res.output_text
```

Step 6: Backend API

Your backend needs two routes:

- /ingest builds the knowledge base index from your PDF
- /chat answers questions using retrieval plus generation

main.py

```python
@app.post("/ingest")
def ingest():
    global index, chunks
    text = pdf_to_text(PDF_PATH)
    chunks = chunk_text(text)
    build_and_save_index(chunks, INDEX_PATH, META_PATH)
    index, chunks = load_index(INDEX_PATH, META_PATH)
    return {"status": "ok", "chunks": len(chunks)}

@app.post("/chat")
def chat(payload: ChatIn):
    global index, chunks

    if index is None or chunks is None:
        if os.path.exists(INDEX_PATH) and os.path.exists(META_PATH):
            index, chunks = load_index(INDEX_PATH, META_PATH)
        else:
            return {"answer": "Knowledge base not ingested yet."}

    hits = retrieve(payload.message, index, chunks)
    answer = generate_answer(payload.message, hits)
    return {"answer": answer}
```

Step 7: Build Frontend

Task for you: Build a small UI using React that opens, shows messages, and sends the user's question to your backend.

Don't worry if you want code, just go to link in bio or comment "RAG".

Comment "RAG" to get the step-by-step guide in your DMs